

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2023.0322000

Decentralized Consensus in robotic swarm for Collective Collision and Avoidance

Yang FengYing^{1,2}, Ahmad Din^{2,3}, Liu HuiChao¹, Mohammad Babar⁴, and Shafiq Ahmad⁵

¹School of Computer and Artificial Intelligence, Huanghuai University, Zhumadian, Henan, 463000, China

²Department of Computer Science, COMSATS University Islamabad, Abbottabad Campus, 22060, Pakistan

³FAST National University of Computer and Emerging Sciences, Islamabad Campus

⁴Department of Computer Science, Abbottabad University of Science and Technology, Pakistan

⁵Industrial Engineering Department, College of Engineering, King Saud University, P.O. Box 800, Riyadh 11421, Saudi Arabia

Corresponding author: Mohammad Babar (drbabar@aust.edu.pk).

“This research has received funding from King Saud University through Researchers Supporting Project number RSP2023R387, King Saud University, Riyadh, Saudi Arabia, the Natural Science Foundation of Henan Province of China(No. 202300410283), the Science and Technology Program of Henan Province of China(No. 202102210336) and the Key R&D Program in Higher Education of Henan Province of China(No. 20B520018).”

ABSTRACT In this paper, a decentralized consensus algorithm for a robotic swarm is presented, which enable agents to escape collisions and avoid obstacles collectively. To achieve consensus, we have used neighboring clusters and extreme function for data dissemination and consensus in agents to avoid obstacles respectively. The main challenge is to make fast and accurate collective decision by improving data propagation in presence of Byzantine agents. To improve data dissemination in local interactions of agents in decentralized fashion, an iterative rumor-based data propagation model is proposed. Due to presence of Byzantine robots, the LCP and the W-MSR algorithm cannot achieve consensus for obstacle avoidance in artificial swarms. We establish the Expectation-based Extreme Value (EEV) algorithm using the local expectation and the extreme function to solve these problems. The experiments conducted in simulations demonstrate that the rumor spreading method has better results than the Peer-to-Peer method in randomly connected swarm signaling network (SSN) with complex environmental circumstance, the EEV algorithm is more effective than the LCP and the W-MSR for the swarm navigation and consensus in agent on large obstacles / environmental features. Furthermore, in presence of malicious / hacked agents in a swarm it is very difficult to reach consensus. The result show that proposed algorithm can handle the Byzantine agents effectively.

INDEX TERMS Swarm Robotics, Consensus Problem, distributed communication, Rumor Spreading, Byzantine Agent, Collective behavior, Obstacle avoidance.

I. INTRODUCTION

CONSENSUS is a far-reaching and challenging topic in artificial and natural swarms, and it is basis for collective decisions, and cooperation in swarm robotics. Various consensus models have been proposed for multi-agent systems in literature [1], [2]. Given the scope of consensus, it can be divided into centralized and decentralized consensus [3]. In the swarm robots with obstacles, the decentralized consensus in the robotic swarm is formed by integrating the opinions of neighbors, rather than referring to the opinions of all individuals in the swarm. For example, the direction and speed of flight of individual birds in the flock are generally determined by reference to nearby neighbors or the observed environmental information.

The swarm intelligence algorithms are applied in many realistic problems to simulate the natural swarms[4], collective motion has critical role in their survival [5]. Such motion is hugely affected by the environmental factors, for instance, gradients, perturbations, obstacles, etc. Therefore, the direction of swarm's movement is determined based on collective sensing. Organisms can enhance their decisions by agglomerating individual estimation [6]. A group of agents estimate predominant distributed environmental feature. It is called collective sensing or collective perception. It has widespread application in collective systems including, to estimate target size [7], block density around constriction site [8], weighted voting[9], best-of-2 problem [10], continuous space problems[11], etc. To estimate target size is very im-

portant in swarms for task allocation.

For the application of the robotic formation more and more widely, under the hypothesis that the communication in the swarm is normal, comparing the data dissemination of the different formation's shapes (random shape, circle shape, rectangle shape), we proposed the advice to the future robotic formation. At the same time, in the application of the robot swarm in the circumstance with the obstacle, we proposed a novel consensus algorithm to avoid the obstacle, it provides a new idea for the consensus study in the swarm with the obstacle. There are two elements to the contribution of this paper. (1) We design and implement a rumor spreading method to accelerate data dissemination. (2) The Expectation-based Extreme Value neighborhood of the agent rumor spreading model is used. For interaction with objects in the environment, for instance obstacle or target, a swarm is needed to decode nature of that unknown object. In case of obstacle, swarm would avoid obstacle (turning swarm) collectively by establishing consensus.

The collective direction of motion to avoid obstacle using popular consensus algorithms, LCP and W-MSR, consensus on the direction of motion is computed by averaging the its own estimation and values received from the neighborhood. It is illustrated that this algorithm is enough to get clear direction to motion to avoid obstacle. In this article, we have introduced a novel aspect of Byzantine robot, which is a Byzantine robot may either hiding its identity or cannot have much impact in its neighborhood. We call such scenario, valid/invalid Byzantine agents. We studied problem that the Byzantine is not valid in some cases. This example is illustrated in Figure 1 and the rules must be kept by the Byzantine robot or each robot in the swarm. To this end, we consider following assumptions: (a) Every agent in the swarm has the ability of the limited perceive. (b)The communication between the robots is reliable in anytime. (c) The Byzantine agent must honestly for itself and it specifies robots which in the honest set.

The remaining paper is organized as follow: Section 2 reviews related work in consensus issues. The models of distributed consensus problem, the data dissemination, the consensus algorithm, and the problem statement are defined in Section 3. In Section 4, the data propagation method and the consensus algorithms are developed. Section 5 discusses the simulation environment and the experimental results. The conclusion and future work are presented in Section 6.

II. RELATED WORK

When Various fruitful research studies have been conducted in last two decades on information transfer / data propagation, task allocation, and consensus achievement in multi-agent systems [12],[13]. Information transfer/ propagation is the precondition to achieve consensus in a swarm [14]. Data can be propagated in a swarm by using explicit or implicit communication mechanisms. Furthermore, the explicit communication mechanism among the agents in swarm has two ways: Peer-to-Peer mode and broadcast mode. In various

studies, communication is considered global or based on computer network communication [15], while many consensus researchers have taken it a default property and ignored it altogether. Moreover, Melnik et al. in [9] discussed the data dissemination process in the blockchain network, and gossip spreading in the swarm. The rumor spreading method in this paper is an improvement of the gossip spreading with the autonomy of the agent and the concurrency of the agent's action execution under the obstacle circumstance. Task allocation is one of the key issue of Multi-agent system, it preallocates a task[12]. When an emergency occurs, it is necessary to adjust task allocation for different scenarios using multi-agent cooperative task reallocation algorithm.

Consensus has a wide spectrum of applications and diverse research issues [16–22], many studies about consensus in multi-agent systems [23–28] or in decentralized consensus [2, 29–33], are valid and important. One of the critical studies in multi-agent systems is a new framework based on the decentralized strategy[2]. The framework include three layers: calculate the force of each robot in a decentralized manner to generic a group, identify a leader for each group by the vision-based goal detection algorithm, using the completely dynamic formation in leader-follower mode to avoid obstacles. The framework is efficiency and reliability by a series experiments. To simplify the interference of time and space in the mobile swarm research, [34] based on algebraic graph theory, matrix theory, and control theory, the network structure of dynamic swarm are abstracted as directed information flow and the topology is fixed or switch. Within this structure, the consensus agreement and convergence are considered. Similarly, the topology of the swarm is also concerned in [35], and [33].

Due to the many works of literature which concentrated only on certain variants of swarm robotics, research has produced studies and can't contact the classification. Valentini et al. [36] proposed the taxonomy method based on the problem or design in 2017. According to the best-of-n problem is symmetric or asymmetric on qualities and costs, four types of study are given, and the case of the asymmetric qualities and asymmetric costs is divided into synergic and antagonistic. In view of the designed methodology, there have two categories: bottom-up and up-down. The paper provides guidance for future research on the consensus classification.

The algorithm or method to solving the consensus has prolific, such as linear consensus protocol (LCP) [34], [35], [37], weight-mean sequence reduced(W-MSR) [35],[37], [38], blockchain [9], [35], [39] smart contract [35], [39], decentralized strategy[2], etc. Saulnier et al. [38] analyze the problem of distributed consensus— all robots are cooperative, but the normal situation is that there are defective or malicious agents in the robotic swarm. To solve this problem, the authors defined the non-cooperative robot and resilient consensus, based on the LCP algorithm, proposed the WMSR algorithm. This algorithm distinguishes the Byzantine agent by removing the maximum or minimum in the sorted list and given the theorem that the resilient asymptotic consensus is

arrival if the number of the swarm is more than $2F+1$ which F is the count of Byzantine robots in the robotics swarm. Base on the W-MSR, Liu et al. in [40] proposed Trust-R to detected the faulty robots. An event triggered update rule is proposed by Wu et al. in [18].

LeBlanc et al. in [39] analyzed the weakness of a typical consensus algorithm that is decided by a global knowledge of swarm robotics, similarly, Valentini et al. in [41] proposed the collective perception scenario and use it to compare the strategies of DMMD, DMVD, and DC through analyzed the weakness of the collective decision-making. So, many researchers focus on decentralized consensus using local knowledge [29], [42–48]. According to the six attributes (name, problem, context, design rational, solution, and case-studies) of a design pattern, Reina et al. in [43] built collective decisions through a cross-inhibition designed model. CDCI is to solving the problem that the agent in the robotic swarm has not the global knowledge of the swarm (such as the population size, the quality of the available alternatives). They used that the Bee colony selects the hive-site as the design principle. The authors used the probabilistic finite state machine to express the state of the agent change and show the application of the macroscopic description and microscopic description scenarios.

Moussaid et al. in [44] proved that the local exchange mechanisms are used to self-organize without any external control operates on the coordinated collective behavior in 2009. Gulzar et al. in [1] given the current state and future direction of the consensus on multi-agent cooperative control in 2018 and proposed that future research should focus on the direct dynamics of cooperative control consensus. A mergeable nervous system is proposed in [49] by Mathews et al. in 2017. Indeed, as can be concluded from the above references well even for that much research build on the theory devise or simulation or using several robots to check the method which is proposed by the authors. While Rubenstein et al. in [50] designed the robot—Kilobot and given the collective algorithm to implement the target shape using the programmable selfassemble with 1024 Kilobots in 2014. The algorithm has three basic collective behaviors: edge-following, gradient information, and localization. These behaviors are implemented by using local interactions and local sensing. The core of these behaviors is the initial group. Through experiment, the authors found “traffic jams” and “erosion” behaviors in the collective movement. The importance of this article is that it has given the new point of the research in the robotic swarm.

III. MODEL

A. DISTRIBUTED CONSENSUS PROBLEM IN ROBOTIC SWARM

We denote swarm of robot with S , which consists of n autonomous robots. The n is an integral, and its value should be greater than 1. We use the term agent to express the robot in the swarm and denote a_i for the i th robot in the swarm $1 < i \leq n$. Swarm robotics is formalized like:

$$S = \{a_1, a_2, \dots, a_i, \dots, a_n \mid n > 1 \text{ and } 1 < i \leq n\}$$

Neighborhood of agent in a swarm is define based on constraint distance D , if the distance between agent a_i and agent a_j is less than D , agent a_j is the neighbor of agent a_i , that is:

$$N_{a_i} = \{a_j \mid \|a_i - a_j\| < D \text{ and } a_i, a_j \in S\}$$

Here, N_{a_i} is the set of the neighbor of agent a_i . $\|a_i - a_j\| < D$ is the Euler distance between agent a_i and agent a_j . $|N_{a_i}|$ is the number of the agent a_i neighbors.

The set of static obstacles in the scenes is denoted by O , and o is the count of the obstacles. Let o_k ($1 \leq k \leq o$) indicates the individual static obstacles. When the agent a_i of the robotic swarm encounters obstacle o_k , the agent a_i will take an opinion of the turn direction to avoid the collision of the obstacle o_k , we denote as follow.

$$v_{a_i}^{o_k}(t) = \begin{cases} -1, & \text{turn right;} \\ \text{or} \\ 1, & \text{turn left;} \end{cases} \quad t = 0; \quad (1)$$

$$f(v_{a_i}^{o_k}(t-1), N_{a_i}^{o_k}(t-1)), \quad t > 0.$$

where the $v_{a_i}^{o_k}$ is used to indicate the opinion of a_i ($a_i \in S$) in the swarm that meets the obstacle o_k ($o_k \in O$). The opinion of agent is computed by the function f based on its current opinion and the latest opinions of its neighbors. Noting that, obstacle o_k should be within the agent a_i 's range of perception. The new opinion is computed based on local information in neighborhood of the agent. The opinion of the swarm S is defined as follow.

$$C_{o_k} = v_{a_1}^{o_k} \cup v_{a_2}^{o_k} \cup \dots \cup v_{a_i}^{o_k} \cup \dots \cup v_{a_n}^{o_k} = \begin{cases} -1, & \text{turn right;} \\ 1, & \text{turn left;} \\ 1, -1, & \text{split.} \end{cases} \quad (2)$$

Here, C_{o_k} denotes the opinion of the swarm encountering the obstacle o_k . Using equation (2), if all of the opinions in the swarm equal to -1, it represents that the swarm turn right. Moreover, if all of the opinions in the swarm equal to 1, the swarm turn left, otherwise, the swarm split two parts, one to the right, the other to the left. The exact consensus on the obstacle o_k is defined by

$$v_{a_i}^{o_k} \equiv C_{o_k} \quad \forall a_i \in S, \exists o_k \in O$$

The exact consensus demands that the opinion of every agent a_i in the swarm must agree with the swarms' opinion when the swarm meets the obstacle o_j . It is obvious that the exact consensus in the swarm is possible for the discrete problem and impossible for the continuous problem. Given the real value of ε as a threshold, the approximate consensus of the obstacle o_k is defined by

$$|C_{o_k} - v_{a_i}^{o_k}| \leq \varepsilon$$

The degree of consensus in the robotic swarm is determined by the value of ε . If the value of ε is lower, the degree of consensus in swarm robotics is higher; whereas, the value of ε is higher, the degree of consensus is lower. When the ε equals 0, the approximate consensus becomes an exact

consensus. The prevailing algorithm for consensus in swarm robotics has the linear consensus protocol and the weighted-mean subsequence reduced algorithm. Then we will discuss both algorithms respectively.

The prerequisite for the completion of the complex task in the swarm is to achieve consensus for the robots. In this work, we study the issue of discrete decentralized consensus in the robotic swarm with obstacles. Furthermore, it is assumed that the swarm has Byzantine robots. The Byzantine agent is honest if the agent is within the honest set, otherwise, the fake value of the Byzantine sends to the agent. Byzantine set is defined by the following.

$$B = \{a_i \mid \begin{array}{ll} v_{a_i}^{o_k} \rightarrow a_j & a_j \in \text{honest}_{a_i} \\ -v_{a_i}^{o_k} \rightarrow a_j & a_j \notin \text{honest}_{a_i} \end{array} \mid a_i \neq a_j\}$$

where $F=|B|$ is the number of the Byzantine set in the robotic swarm. In common cases, F is unknown, for the convenience of computation, many research assumes that the F is known.

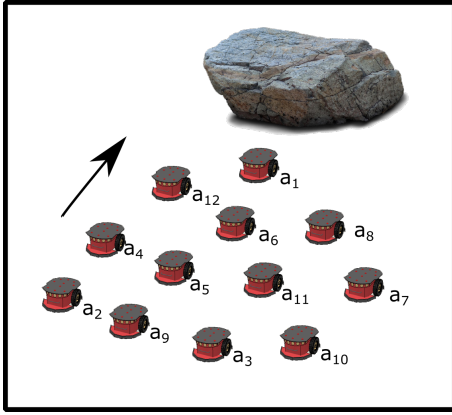


FIGURE 1. The Invalid Byzantine Robot

Figure 1 illustrate an instance of such swarm with Byzantine agent. Here, robot # 6 is a Byzantine robot. The honest set of the robot 6 is $\{1, 5, 12, 8, 11\}$, therefore, robots in set $\{2, 3, 4, 7, 9, 10\}$ will receive the fake value from robot 6. Furthermore, if the robots in the second set are not in neighborhood of robot 6, then it cannot propagate fake values in the swarm. In this case, the Byzantine robot 6 is called an invalid Byzantine agent. Conversely, it is a valid Byzantine agent. From the experimental results, we discovered that a Byzantine robot being invalid is not uncommon.

B. DATA DISSEMINATE

The Peer-to-Peer data dissemination method is very popular in robotic swarm. In this method transfer speed is low because the propagation process is linear. On other hand, the rumor spreading has the characteristics that it spreads quickly, the breadth of wide, but it can easily turn into the rumor storm due to exponential increase of the rumor. Considering the autonomy and concurrency of the agent in the swarm, some researchers proposed other data dissemination techniques. Melnik et al. involved the gossip data dissemination [8].

Based on the gossip data dissemination, we have perfect and improve it to construct the rumor spreading method.

The rumor spreading method includes the neighboring cluster, the receiving agent, and the rumored couple. The neighboring cluster is defined as follows. The parameter t is the evaluation times in the formula.

$$G_{a_i}(t) = \begin{cases} \{a_i\}, & t = 0; \\ G_{a_i}(t-1) \cup (\bigcup_{a_j \in R_{a_i}(t-1)} N_{a_j}), & t > 0. \end{cases} \quad (3)$$

where N_{a_j} is the neighbor set of the agent a_j . $R_{a_i}(t-1)$ is the set of the receive agents. The receiving agent is defined as below.

$$R_{a_i}(t) = \begin{cases} G_{a_i}(t), & t = 0; \\ G_{a_i}(t) - G_{a_i}(t-1), & t > 0. \end{cases} \quad (4)$$

The data disseminate couple as follows.

$$C_{a_i}(t) = \begin{cases} I \longrightarrow R_{a_i}(t), \text{ for } t = 0; \\ a_k \longrightarrow a_j, \text{ for } \forall a_j \in R_{a_i}(t) \text{ and } \\ a_j \in \bigcup_{a_k \in R_{a_i}(t-1)} N_{a_k} \text{ and } t > 0. \end{cases} \quad (5)$$

here, I denotes the data to be disseminate. The $\longrightarrow$ denotes the data from a_k to a_j . An example of a description of the rumor spreading in Figure 2. The Figure 2(a) illustrates the initial state ($t=0$) in which a_1 perceives the obstacle and makes an announcement I , that is the obstacle in front of the swarm movement. Now, $G_{a_1}(0)=\{a_1\}$, $R_{a_1}(0)=\{a_1\}$, $C_{a_1}(0)=\{I \longrightarrow a_1\}$. Figure 2(b) is the rumor spreading in $t = 1$. Agent a_1 detected its neighbors around the 0.5m range and found that a_{12} and a_6 are its neighbors. So, $G_{a_1}(1)=\{a_1, a_{12}, a_6\}$, $R_{a_1}(1)=\{a_{12}, a_6\}$, and $C_{a_1}(1)=\{a_1 \longrightarrow a_{12}, a_1 \longrightarrow a_6\}$, i.e., a_1 sends the rumor I to its neighbors (a_6, a_{12}). Figure 2(c) shows the second rumor spreading ($t=2$). At this step, a_{12} and a_6 detected its neighbors respectively. Agent a_{12} has the neighbors $N_{a_{12}}=\{a_1, a_5\}$, while the neighbors of a_6 is $N_{a_6}=\{a_1, a_{11}\}$. Then, $G_{a_1}(2)=\{a_1, a_{12}, a_6, a_5, a_{11}\}$, $R_{a_1}(2)=\{a_5, a_{11}\}$, and $C_{a_1}(2)=\{a_{12} \longrightarrow a_5, a_6 \longrightarrow a_{11}\}$. Figure 2 (d),(e), and (f) are similarly with above process. At the end of the rumor spreading process, $G_{a_1}(5)=\{a_1, a_{12}, a_6, a_5, a_{11}, a_2, a_9, a_{10}, a_7, a_3, a_8, a_4\}$, i.e., the G equals to the swarm N .

Note that the data disseminate couple is not unique. Suppose that agent a_{12} has the neighbors $N_{a_{12}}=\{a_1, a_6, a_5\}$, while the neighbors of a_6 is $N_{a_6}=\{a_1, a_{12}, a_5, a_{11}\}$ in Figure 2(c), that is a_5 could receive the rumor I from a_{12} or a_6 .

The rumor spreading method comes to end when one of the two conditions emerge: (i) When the number of agents in the neighboring cluster G equals the swarm, the method ends. (ii) There is no new neighbor for the neighboring cluster but its number is no equal to the swarm, the method also ends. From the first condition, we could get that the graph of the swarm is connected. While, second condition indicates that the graph of the swarm is disconnected.

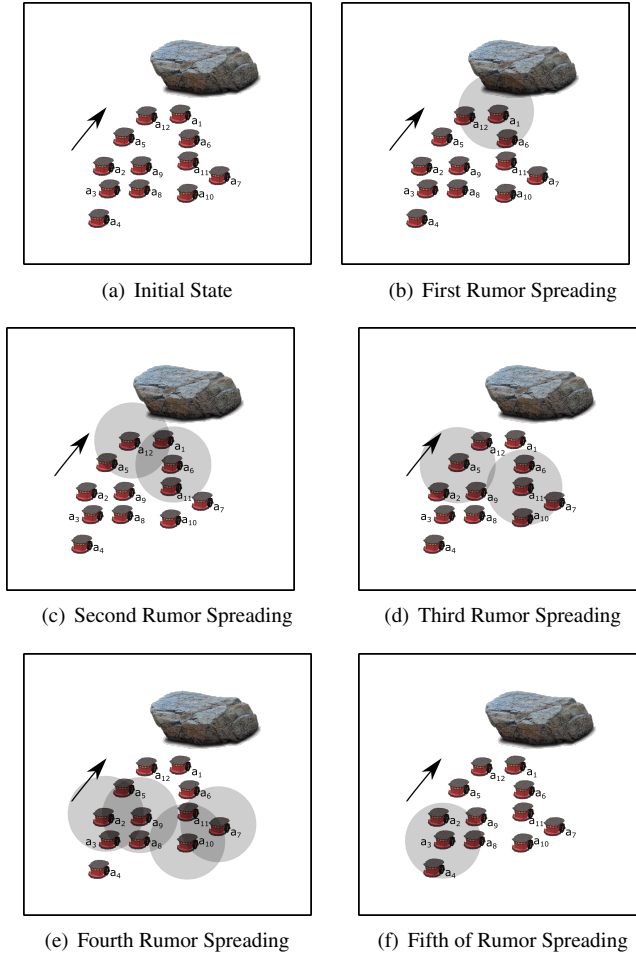


FIGURE 2. Process of the Rumor Spreading

C. LINEAR CONSENSUS PROTOCOL

The simplicity and effectiveness of the linear consensus protocol (LCP) attracted the attention of numerous researchers. The basic spirit of the LCP is that the values of the current agent and its neighbors are calculated using the average value method. The LCP of the robotic swarm is defined as follow:

$$v_{a_i}^{o_k}(t+1) = w_{a_i}^{o_k}(v_{a_i}^{o_k}(t) + \sum_{a_j \in N_{a_i}} v_{a_j}^{o_k}(t)) \quad t > 0 \quad (6)$$

where $v_{a_i}^{o_k}(t+1)$, $v_{a_i}^{o_k}(t)$ are the opinion of agent a_i over time t and $t+1$. The $w_{a_i}^{o_k}(t+1)$ is the linear coefficient of the equation and adjusts the influence for the agent a_i by its neighbors. $v(a_i, o_k)(0)$ takes the original value using (1). In general, the weight is $w_{a_i}^{o_k} = \frac{1}{|N_{a_i}|+1}$, and $(|N_{a_i}|+1) \cdot w_{a_i}^{o_k} = 1$. The essence of the LCP algorithm is to compute the average of these values of the agent a_i and its neighbors. LCP algorithm can achieve consensus in some cases. When the scene has the Byzantine agents in the robotic swarm, LCP has no deal for these Byzantine agents. In this case, the consensus is invalid. The resolution of the issue could use the weighted-mean subsequence reduced as follows.

D. THE WEIGHTED-MEAN SUBSEQUENCE REDUCED ALGORITHM

The weighted-mean subsequence reduced (W-MSR) algorithm was proposed by LeBlanc et. al in [38]. This is an improvement for the LCP algorithm. Similarly for LCP, the agent a_i 's consensus is decided by itself and its neighbors, but the process is different. Given the parameter F that expresses the number of Byzantine agents, the W-MSR algorithm is divided into three stages.

First, sort the values of agent a_i and its neighbors. Agent a_i receives the copy values of its neighbors and sorts them in ascending order, form the sorted list A_{a_i} . Secondly, tag the possible Byzantine agent to the sorted list A_{a_i} . Agent a_i compares the current value $v_{a_i}^{o_k}$ to every value in the sorted list A_{a_i} . If there are fewer F values in sorted list A_{a_i} smaller than $v_{a_i}^{o_k}$, then delete all values that smaller than $v_{a_i}^{o_k}$ and register these agents in the list of Byzantine agent B_{a_i} . If there are F or more values smaller values than $v_{a_i}^{o_k}$, the F smallest values are deleted and record it in the list of Byzantine agents B_{a_i} . The process is similar to the values in the sorted list A_{a_i} larger than $v_{a_i}^{o_k}$. Third, calculate the new value of agent a_i ($a_i \notin B$) for the obstacle o_k . Agent a_i updates its values as follows:

$$v_{a_i}^{o_k}(t+1) = w_{a_i}^{o_k}(v_{a_i}^{o_k}(t) + \sum_{a_j \in (N_{a_i}-B_{a_i})} v_{a_j}^{o_k}(t)) \quad t > 0 \quad (7)$$

here, $w_{a_i}^{o_k} = \frac{1}{|N_{a_i}|+1-|B_{a_i}|}$, and $|B_{a_i}|$ is the number of the Byzantine set B_{a_i} to agent a_i .

In W-MSR algorithm, the non-Byzantine agent updates its value using itself and its non-Byzantine neighbors. The W-MSR algorithm detects the possible Byzantine agents through the comparison the value with its value. The advantage is that it reduces communication costs. However, it has three disadvantages. First, there is an inaccurate consensus. Because the W-MSR algorithm cannot determine exactly which agent is the Byzantine agent, the new value of the agent is computed using the values of blurry neighbors honestly. Second one is that when this case in Section 1 occurred, this Byzantine agent is an invalid Byzantine, the W-MSR algorithm does not know this and calculates the new value use its method. The last one is that this algorithm depends upon the F parameter to work. For these problems, we propose the expectation-based extreme value algorithm.

E. THE EXPECTATION-BASED EXTREME VALUE ALGORITHM

The idea of the Expectation-based Extreme Value algorithm (EEV) is to calculate the agent's expectation value E through the values of the agent and its neighbor agents. Then agent's new value depend on the expectation E through an extreme function. The expectation value E is calculated as followings.

$$E_{a_i}^{o_j}(t) = \begin{cases} 0, & t = 0, \\ \frac{1}{|N_{a_i}|+1} (v_{a_i}^{o_j}(t) + \sum_{a_j \in N_{a_i}} v_{a_j}^{o_j}(t)), & t > 0. \end{cases} \quad (8)$$

The extreme function shown is this.

$$v_{a_i}^{o_j}(t+1) = \left\{ \begin{array}{ll} 1, & \text{if } E_{a_i}^{o_j}(t) > 0; \\ v_{a_i}^{o_j}(t), & \text{if } E_{a_i}^{o_j}(t) = 0; \\ -1, & \text{if } E_{a_i}^{o_j}(t) < 0. \end{array} \right\} \quad t \geq 0 \quad (9)$$

Here, the initial value is same as equation (1). When the expectation is greater than 0, it shows that plural agents around the agent a_i turn left, the new value of the agent a_i take 1, i.e., turn left. When the expectation equals to 0, it illustrates that the number to turn right equals to turn left, in this case, the new value of the agent a_i keeps the original value. When the expectation is lower than 0, it shows that plural agents around the agent a_i are turning right, the new value is -1. In essentially, $v_{a_i}^{o_j}(t+1)$ only takes one of the extreme value (-1/1) or keep the original value, so the function is called extreme function.

F. PROBLEM STATEMENT

In this section, we present the issue of study in our work in the form of Figure 1. The direction of the arrow corresponds with the direction of the swarm movement. As the swarm moves forward, the agent in the head swarm has found the obstacle. How to avoid the obstacle? For the ground mobile robots, there have three strategies to get around the obstacle: turn right, turn left, or split into two-part. The turning swarm has three problems: (1) data disseminated, (2) consensus achievement, (3) identify valid/invalid Byzantine.

(1) Data disseminate. How to spread the obstacle message to all the agents in the swarm? It has many methods such as Peer-to-Peer liner propagation, broadcast spreading, etc. Which method is better? This is the first study issue in this paper.

(2) Consensus achievement. LCP and W-MSR are the widely used consensus algorithms. They get the new value by the mean value method. The result is inaccurate or invalid in certain cases. The way to solve the problem is the second problem in this work.

(3) Identify valid/invalid Byzantine. The W-MSR cannot identify which is a Byzantine agent, that is, the agent cannot known the received values that are true or fake. These causes three cases: all neighbors of the Byzantine agent receive the true value and the fake value is invalid for the non-neighbors, all neighbors of the Byzantine agent receive the fake value and the true value is invalid for the non-neighbors, the partial neighbors of the Byzantine agent receive the true value and partial neighbors receive the false value. The Byzantine agent is an invalid Byzantine agent in the first case, this is a work on other cases. How to identify valid/invalid Byzantine agents is the final issue.

IV. DESIGN ALGORITHM

In this section, we have outlined for data dissemination and consensus algorithms. Furthermore, variables used in these algorithms are described as well. The variable ot is the obstacle message that the swarm met, for convenient operation, we take the obstacle id. Similarly, each agent is represented by

a variable $agent$, which is an agent id. The variable n is the number of the agents in the swarm. The variable $neighbor$ is the neighbor's set of the robots in the swarm. The variable $opinion$ is the set of agent's opinion for the obstacle.

A. DATA DISSEMINATION ALGORITHM

For the data dissemination, we generate 110 scenes randomly to compare the Peer-to-Peer and the rumor spreading methods. Each scene is built by 100 ~ 1100 agents. If the scenes have the same amount of the agents, they have different shapes, that is they have the different formation. The description of the two algorithms is show as follows. Peer-to-Peer method is a simple, series-based method.

Algorithm PP : Peer_to_Peer($ot, agent, n, neighbor$)	
1:	Initialize: $\{message = (-1, -1, \dots, -1)_n; swarm = (0, 1, \dots, n-1); i = agent\}$
2:	set $message(i) = ot$
3:	delete i from the $swarm$
4:	while $swarm \neq \text{NULL}$:
5:	if $\text{len}(neighbor(i)) \neq 0$:
6:	set $i = \text{random select one from } neighbor(i)$
7:	delete i from $neighbor(i)$
8:	else:
9:	set $i = \text{random select one from } swarm$
10:	end if
11:	delete i from $swarm$
12:	set $message = ot$
13:	end while
14:	return $message$

The rumor spreading method consists of four parts: generated the neighboring cluster, decide the receiving agent, build the rumored couple, and propagated the rumor.

Algorithm RS : Rumor Spread($ot, agent, n, neighbor$)	
1:	Initialize: $\{swarm = (0, 1, \dots, n-1); g0 = \{agent\}; g1 = \{\}; r0 = g0; r1 = \{\};\}$
2:	while $swarm \neq \text{NULL}$:
3:	calculate the $g1$ using the formulation (3)
4:	if $g0 \neq g1$
5:	select one element x from the $g1$
6:	if x 's $neighbor$ exist in $swarm$
7:	set $g0 = x$
8:	else:
9:	set $g0 = \text{select one element } x \text{ from the } swarm$
10:	calculate the $r1$ using the formulation (4)
11:	calculate C and spread the ot to the $r1$
12:	set $g0 = g1$
13:	set $r0 = r1$
14:	delete all the agents in $swarm$ that had received ot in $r1$
15:	end while

B. CONSENSUS ALGORITHM

In consensus algorithms, we compare the LCP, W-MSR, and EEV algorithms in four scenes under in CoppeliaSim using 30 Pioneer_3dxes mobile robots. The implication of each algorithm as follows. Note that -1 refers to turning right, 1 refers to turning left. The core of the LCP algorithm is to calculate the following value for each agent using the mean method.

There are two aspects to the W-MSR kernel: delete the extreme value after sorting the current value, calculate the new value for each 'honest' agent.

The primal elements of the EEV algorithm have a cycle comparison to identify the Byzantine agent and calculate the new value for the honest agent base on the piecewise function of the local opinions. This algorithm describes as follows.

V. RESULTS

Algorithm LCP : LCP($n, opinion, neighbor$)

```

1: Initialize: { $max = -100, min = 100$ }
2: while  $|max - min| \geq 1e-2$ :
3:   set  $temp = NULL$ 
4:   for  $i=0$  to  $n-1$ :
5:     set  $s=0$ 
6:     for  $j$  in  $neighbor(i)$ :
7:       set  $s = s + opinion(i)$ 
8:     end for  $j$ 
9:     set  $temp = s/|Ni|$ 
10:   end for  $i$ 
11:   for  $i=0$  to  $n-1$ :
12:     set  $opinion(i) = temp(i)$ 
13:     if  $max < opinion(i)$ : set  $max = opinion(i)$ 
14:     if  $min > opinion(i)$ : set  $min = opinion(i)$ 
15:   end for  $i$ 
16: end while
17: return  $opinion$ 

```

Algorithm W-MSR : WMSR($n, F, nb, opinion, b$)

```

1: Initialize: { $swarm = (0, 1, \dots, n-1)$ ;  $p = random(0 \sim n-1, n/2)$ ;  $sum = 0$ ;  $max = -100, min = 100$ }
2: while  $|max - min| \geq 1e-2$ :
3:   the agent spread its opinion to other agents in the swarm, if the agent in set  $b$ , spread its truth opinion to the agent in set  $p$  and send the reverse opinion to other agents that are not in set  $p$ . Each agent copies the opinion as an opinion list.
4:   Sort the opinion list by ascending for each agent in the swarm.
5:   Delete  $F$  opinions greater than the current agent's opinion, if the count of opinions greater than itself less than  $F$ , delete all of greater than itself, or delete  $F$  opinions less than current agent's opinion, the count of opinions less than itself fewer  $F$ , then delete all of less than itself.
6:   Compute the new opinion for the honest agent use the formula (5) using the reminder opinion.
7:   Get the max opinion and min opinion through comparison.
8: end while
9: return  $opinion$ 

```

Algorithm EEV : EEV($n, nb, opinion, b$)

```

1: Initialize: { $avg = (0, 0, \dots, 0)$ ;  $p = random(0 \sim n-1, n/2)$ ;  $sum = 0$ ;  $curr\_opinion = ((-2, -2), \dots, (-2, -2))_n, \dots, ((-2, -2), \dots, (-2, -2))$ }
2: Same to WMSR's 2rd step and save opinion to  $curr\_opinion$ 
3: for  $i=0$  to  $n-1$ :
4:   for  $j=0$  to  $n-1$ :
5:     for  $k=0$  to  $n-1$ :
6:       if  $curr\_opinion(i, k, 1) = -2$  and  $curr\_opinion(j, k, 1) = -2$  and  $curr\_opinion(i, k, 1) = curr\_opinion(j, k, 1)$  and  $k$  not in  $r$ :
7:         append  $k$  to  $r$ 
8:       end if
9:     end for  $k$ 
10:   end for  $j$ 
11: end for  $i$ 
12: Delete all agent in set  $r$  from  $curr\_opinion$ 
13: while  $m=1$ :
14:    $m=0$ 
15:   compute the expectation of each agent using  $curr\_opinion$  and  $nb$ , save the result to  $avg$ .
16:    $pnew = (0, 0, \dots, 0)_n$ 
17:   for  $i=0$  to  $n-1$ :
18:     if  $avg(i) > 0$ : set  $pnew(i) = 0$ 
19:     else if  $avg(i) = 0$ : set  $pnew(i) = opinion(i)$ 
20:     else: set  $pnew(i) = -1$ 
21:   end for  $i$ 
22:   for  $i=0$  to  $n-1$ :
23:     if  $opinion(i) = pnew(i)$ :
24:       set  $opinion(i) = pnew(i)$ ;  $m=1$ 
25:   end for  $i$ 
26:   spread the new  $opinion$  of each agent to its neighbors
27: end while
28: return  $opinion$ 

```

A. SETUP OF THE SIMULATION

The experiments were conducted on Intel(R) Core(TM) i5-4300U processor with 8GB RAM in 64bit Windows 7 operating system. The programs of the data dissemination are written in Python and consensus algorithm were also implemented using Python as well. To compare the two methods of data dissemination, we build basic scenes with 100 ~ 1100 simulated agents. To this end, the simulated agents, were defined the different shapes and constraint conditions to compare the difference between the Peer-to-Peer method and the rumor spreading method.

To compare the three consensus algorithms, we have used the same scene with the 30 Pioneer_3dxes with a large obstacle, and 100 clusters of opinions for the robotic swarm. Each one runs 100 times with these opinions and arrives at a consensus if the error of the maximum point and the minimum point is less than 0.01. In the scene, we suppose 6 cases: no Byzantine swarm(SCENE1), the swarm with 4 Byzantine agents(SCENE2_1), the swarm with 8 Byzantine agents(SCENE2_2), the swarm with 12 Byzantine agents(SCENE2_3), SCENE3 is the case of the swarm with 20 Byzantine agents, the swarm with unknown number of Byzantine agents(SCENE4).

Furthermore, in order to prevent the case in which Byzantine agent always be honest or cheat at certain agents in the

swarm, we use the stochastic method to produce the honest set every time. The Byzantine agent cheats to the agent in the honest set, while it doesn't cheat agents not in honest set. In additional, on the cost times and loop times, we use some parameters to verify the performance of the algorithm as follows.

Average cost time(A_t): This parameter indicates the running of algorithm in seconds. Lower values indicate better for the algorithm.

$$A_t = \frac{1}{M} \sum_{i=1}^M (t_i), t_i > 0. \quad (10)$$

Here, M is the running times of the algorithm, t_i is the cost time to i th running time of the algorithm.

Standard deviation of the cost time(S_t): This parameter shows the time variation of the data dissemination method.

$$S_t = \sqrt{\frac{(t_1 - A_t)^2 + (t_2 - A_t)^2 + \dots + (t_N - A_t)^2}{M}} \quad (11)$$

Here, the parameters same to above.

Average loop times(A_l): This parameter indicates the loop times of the consensus algorithms in times.

$$A_l = \frac{1}{M} \sum_{j=1}^M (l_j), l_j > 0. \quad (12)$$

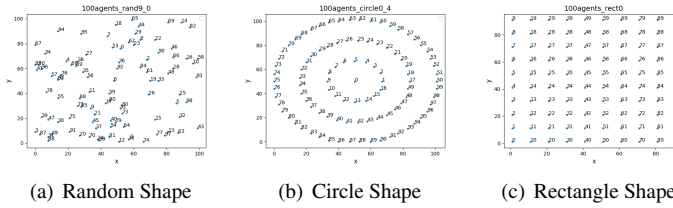


FIGURE 3. Shapes of the Data Dissemination

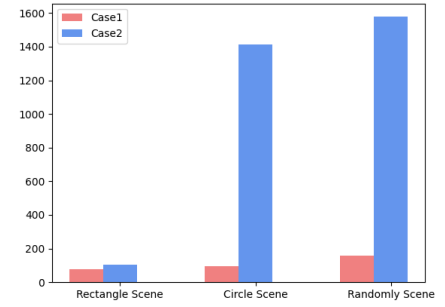


FIGURE 4. Results of the Data Dissemination

Here, l_j is the loop times of the algorithm.

B. RESULTS, DISCUSSION, AND INTERPRETATION

In this section we present the results of the experiments and discuss results.

1) Data dissemination

To compare the Peer-to-Peer method and Rumor Spreading method, we have designed three dissemination shapes, including, random shape, circle shape, and rectangle shape. In random form, we randomly generated 110 basic-scene with 100 ~ 1100 agents. For each basic-scene, 8 to 9 different neighbor-scenarios were generated depending on the different constraint distance D with respect to the neighbors. Due to the randomness of the data dissemination, each experiment for neighbor-scenarios were executed 10 times for both Peer-to-Peer method and the Rumor Spreading method. Final results were computed by averaging data collected for neighbor-scenarios respectively. Finally, there were 1783 different results.

In circle shape, we randomly generated 11 basic-scenarios which same to above. Each basic-scenarios was divided into 8 layer-scene levels based on the 2 ~ 9 radius to decide the concentric circle. In the same way, each layer-scene is divided into 8 or 9 neighbor-scene in the constraint distance D . Consequently, we obtained 1506 different out-comes.

Like the shape of a circle, in rectangle shape, we gave the constant distance between the agents and using the above method. As a result, 178 different results were produced. The shapes of the data dissemination are show in Figure 3.

In the experiments, the starting point of data propagation is the agent nearest to the origin. The results of data dissemination are shown in Figure 4. The results are divided into two types based on the cost required for data dissemination using rumor spreading method and the Peer-to-Peer method. If the cost time of the rumor spreading method is greater than or equal to Peer-to-Peer, we have called Case1, otherwise, called Case2. From the results, we could get the proportion of the Case2 occurred of the total number in Randomly scene, Circle scene, Rectangle scene are 0.908, 0.938, 0.573 respectively. Obviously, the rumor spreading is better than the Peer-to-Peer on the majority of scenes, but it degenerates quickly when meeting a strictly equal distance between the agents, such as the rectangle shape. The main reason is that the equal number of neighbors between all of the agents causes the agent to have

no more choices on the data dissemination. It cannot use the advantage of the rumor spreading.

The statistical results of the data dissemination are shown in Figure 5. Figure 5(a) gives the average(A_t) of the cost time of the Rumor Spreading method and Peer-to-Peer method. We found that maximum variation in three scenes for Peer-to-Peer method (Case1) is 0.0173. Similarly, the maximum variation of the rumor spreading is 0.0138. However the cost time of the Peer-to-Peer is quickly increases in Case2, and it decreases for rumor spreading method. This dictates that the performance of rumor spreading method is approximately similar to the Peer-to-Peer method when the rumor spreading method degenerates in some scenes, such as the rectangle-scene, however, the rumor spreading method is more stable compared to the Peer-to-Peer method.

There are again verified in Figure 5(b) that the differences between rumor spreading method and Peer-to-Peer method—the standard deviation (S_t) of the cost time. In this figure, the values of the standard deviation are less than 0.02 except the Peer-to-Peer under Case2. We get the conclusion that the rumor spreading method is a stable method.

In the data dissemination speed, the maximum difference in terms of the cost time for the Case1 with same scenarios in Circle scene was 0.013001 in case of Peer-to-Peer and 0.116006 in case of Rumor Spreading. Moreover, in Rectangle scene cost time for Peer-to-Peer was 4.893280 and 0.064004 for Rumor Spreading in Case2. The highest error for rumor spreading method is 8.9231 times compared to the Peer-to-Peer method in Case1. However, the highest error for Peer-to-Peer method is 76.4531 times compared to rumor spreading method in Case2. Therefore, the rumor spreading method has high speed compared to the Peer-to-Peer method in more than 90 percent of cases.

For the individual scenes result of cost time for comparable for both Rumors Spreading method and Peer-to-Peer method. However, the parallel propagation of opinion among autonomous agents in theory and real application, the propagation speed of the Rumor Spreading method is faster than the Peer-to-Peer method. Note: Rumor Spreading method has certain disadvantage, for instance, it does not guarantee that the path of the data dissemination is shortest.

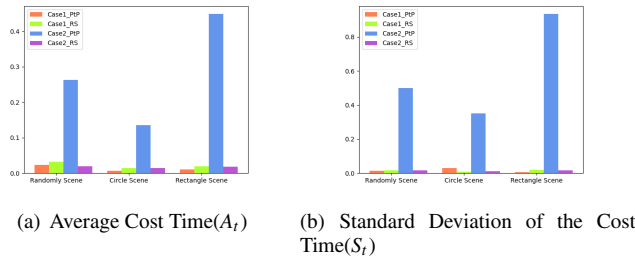


FIGURE 5. Statistical Results of the Data Dissemination

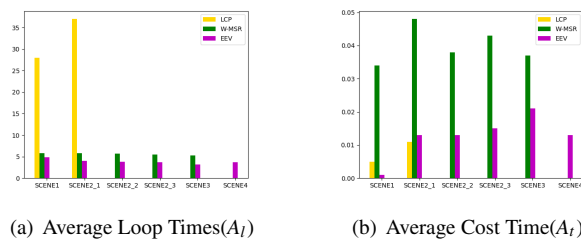


FIGURE 6. Results of the Consensus Algorithms

2) Consensus algorithm

The results of the consensus algorithms are shown in Figure 7. The simulation has been run 100 times for each scene in consensus algorithm. We have compared the algorithms from two aspects, including, average loop times(A_l) and average cost time(A_t). All of the algorithms were arrived to the consensus according to demand. Based on the results, we could able to obtain the following analysis from the Figure 6(a) and Figure 6(b).

In the SCENE1, the LCP algorithm has the highest loop times, because this algorithm has high fluctuations. However, its structure is simplest, its cost time is lower compared to the W-MSR algorithm. The W-MSR algorithm has more complicated processing, therefore, its loop times are lower, but its cost time is highest in all three algorithms. The EEV algorithm has a simple structure than W-MSR, hence its loop times and cost time are lower than LCP and W-MSR. The mean cost time(A_t) for the W-MSR algorithms is 40 times the EEV algorithm.

In the SCENE2_1, the cost time of the EEV algorithm and W-MSR algorithm increases rapidly. The cause of this rapid rise is the due to the Byzantine agents. The W-MSR use the sorting, while the EEV uses the comparison. For the W-MSR, the current cost time is 1.4308 times higher compared to the SCENE1. For the EEV, its cost time in this SCENE is 13.6202 times higher compared to the SCENE1. The cost time of LCP is stable because it is no handling Byzantine agents, therefore, it is equal to the SCENE1. However, its loop times increase quickly compared to the other algorithms. The reason behind higher loop time is Byzantine agents which prevents algorithm from achieving consensus. Based on the

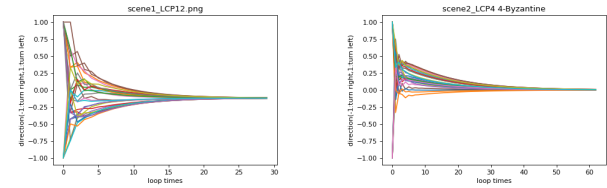


FIGURE 7. Results of the LCP

average cost, the average cost time(A_t) of the W-MSR is 4.2184 times higher compared to the LCP. Furthermore, the average cost time(A_t) of the EEV is comparable to the LCP.

As the LCP algorithm does not handle Byzantine agents, therefore, we are not considering the LCP algorithm hereinafter. In SCENE2_2, SCENE2_3, and SCENE3, the mean loop times(A_l) for W-MSR are all more than 5.0, while the average loop times(A_l) of the EEV are all less than 4.0. The average loop times(A_l) of W-MSR are 1.49, 1.5, 1.64 times compared to EEV in SCENE2_2, SCENE2_3, and SCENE3 respectively. Likewise, the average cost time(A_t) of W-MSR is 3.00, 2.89, 1.76 times compared to the EEV in those scenes respectively. It is obvious that when the number of the Byzantine agents increases, the Byzantine agents affect the W-MSR higher compared to the EEV, i.e., the EEV has the stable feature in this case. From the average cost time(A_t), when the number of Byzantine agents increases, the honest agents' decreases, therefore, the cost time difference between W-MSR and EEV is decreasing with the honest agents.

The SCENE 4 in Figure 6 shows the result of EEV in presence of unknown the number of the Byzantine agents in the swarm. In this experiment to check the effectiveness of EEV, all the Byzantine agents were generated randomly. The range of the number of Byzantine agents is between 1 and 20, the average is 9.73. From the results, EEV is robust for the unknown number of the Byzantine agents and the independent feature of the F parameter.

In sum, the simple structure of the LCP algorithm could quickly reach consensus in the non-Byzantine swarm. Due to it cannot handle Byzantine agent, hence it can potentially use the fake opinion of Byzantine to the consensus, it cannot be used for the swarm with the Byzantine. The W-MSR algorithm could also achieve consensus in each SCENE except SCENE4. Its loop times are shorter and stable, but its cost time is higher than LCP and EEV algorithm. The EEV works high efficiently of on each scene. The major issue is that the LCP and W-MSR consensus value is not valid for our obstacle avoidance in swarm. From this result, all LCP and W-MSR consensus results are near zero, only EEV consensus value is 1 or -1, i.e., EEV algorithm can solve the turning swarm. The special outcomes of consensus algorithms are shown in Figure 7 ~ 10.

Figure 7 shows the two results of the LCP algorithm for SCENE1 and SCENE2_1 respectively. The figure on the left is the result of the experiment 12 without Byzantine. The loop times is 29 and the consensus value is -0.11904. The right one

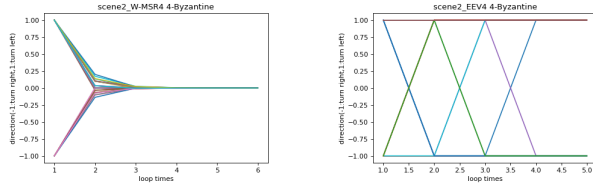


FIGURE 8. Results of the 4-Byzantine

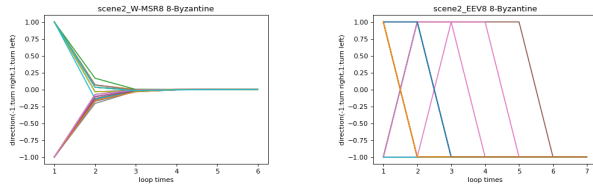


FIGURE 9. Results of the 8-Byzantine

is the experiment 4 with 4-Byzantine agents. The loop times is 62 and the consensus value is 0.003982. The results of the experiment 4 with 4-Byzantine agents are shown in Figure 8. The picture on the left uses the W-MSR algorithm. Its loop times is 6 and the consensus value is 0.00005143. The right one is the result of the EEV algorithm. Its loop times is 5 and the consensus value is that 21 agents equal to 1(turn left) and 9 agents equal to -1(turn right), which is the swarm divided into two parts in this case.

Figure 9 is the result of the experiment 8 with 8-Byzantine. The left one uses the W-MSR algorithm. Its loop times is 6 and the consensus value is -0.0000837. The right one is the result of the EEV algorithm. Its loop times is 7 and the whole consensus value is -1, which means that all agents in the swarm turn right.

The results of experiment 56 with 12-Byzantine are depicted in Figure 10. As above, the figure on the left is based on the W-MSR algorithm. Its loop times is 5 and the consensus value is 0.000139. The one on the right is based on the EEV algorithm. Its loop times is 5 and the consensus value are 1, i.e., the swarm turn left.

In all results explained above, the EEV algorithm has a valid direction to the swarm with the obstacle, but the LCP or W-MSR have an undecided direction.

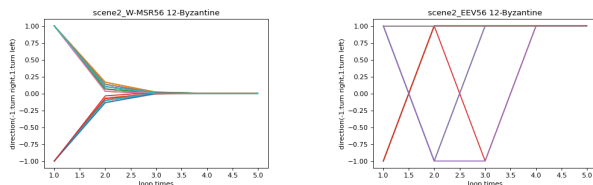


FIGURE 10. Results of the 12-Byzantine

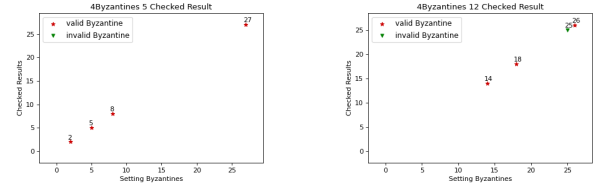


FIGURE 11. Results of SCENE2_1

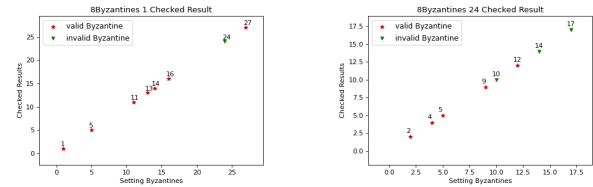


FIGURE 12. Results of SCENE2_2

3) Identity valid/invalid Byzantine

One of the outcome of the EEV algorithm is the identity of the valid and invalid Byzantine agents. The particular results of identification in Byzantine were stochastic selected and shown in the Figure 11 ~ 14.

Figure 11 shows the swarm with 4 Byzantine agents. The figure on the left is generated by experiment 5, while the right one is generated by experiment 12. All the Byzantines work in experiment 5, but agent 25 is an invalid Byzantine agent in experiment 12.

The results of SCENE2_2 are shown in Figure 12. The left figure with (1, 5, 11, 13, 14, 16, 24, 27) Byzantine agents have an invalid Byzantine(agent 24) in experiment 1, but the one on the right has three invalid Byzantines (10, 14, 17) and others are valid Byzantines.

Figure 13 shows that the result of SCENE2_3. Same to above experiments, the left figure is the result of the 51 experiment, and it has 12 Byzantines and all are valid agents. The figure on the right has 4 invalid Byzantines and 8 valid Byzantines. In Figure 14, all of the figures are the result of undetermined Byzantine. The left picture is the result of the experiment 0 in SCENE4. It has 8 Byzantines(1, 7, 9, 16, 17, 19, 20, 24). Agent 7 and 19 are the valid and others are invalid. The right one is the result of the experiment 80 in SCENE4. It has 19 Byzantines(0, 2, 5, 6, 8, 9, 11, 12, 14, 15, 17, 18, 19, 20, 21, 23, 24, 28, 29). Agent 11, 14, 15, 17, 18, 21, 24, 29

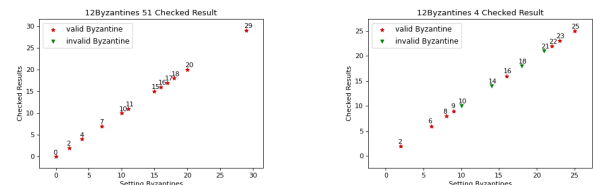


FIGURE 13. The Result of SCENE2_3

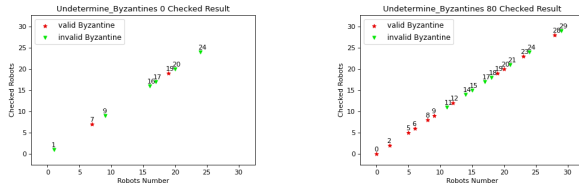


FIGURE 14. The Result of SCENE4

are the invalid Byzantines and others are valid.

From the results presented above we can conclude that the EEV algorithm can identify the Byzantine agent that its part neighbors receive the true opinion and other neighbors receive the fake opinion or all of its neighbors receive the fake opinion. Note that all of the honest set is generated using the random mode, the result of the Byzantine is not constant and deterministic.

VI. CONCLUSION

In this article, our aim was to design decentralized consensus algorithm in presence of Byzantine agents for collective decisions during navigation of swarm in hostile environments. Data dissemination is one of the critical problems in decentralized collective decisions and to achieve consensus. To this end, we presented a rumor spreading method and the Expectation-based Extreme Value (EEV) algorithm. We have compared the rumor spreading method to the Peer-to-Peer method. We designed the extreme function as the update-state function for the agents in the swarm and used it in EEV. We simulated LCP, W-MSR, and EEV algorithms.

The rumor spreading is combined iteratively by the neighboring clusters, the receiving agent, and the rumored couple. It takes advantage of the autonomy and concurrency of the robots in the swarm to obtain rapid data dissemination compared to the Peer-to-Peer method on the stochastic and regular scenes, at same time, it uses the constraint radius to prevent the exponential explosion of the rumor spreading. The EEV algorithm is accomplished by the local expectation and the extreme function. It solves the problems that the LCP and W-MSR algorithms cannot reach the consensus on the navigation and obstacle avoidance in swarms, and the W-MSR algorithm cannot identify the Byzantine agent. From the simulation result, the EEV algorithm can be validated that the Byzantine agent works or not.

In future, we have plan to improve this work in following directions. The rumor spreading method had found that the shape of the swarm has effect the data dissemination. we would like to quantify its effect. Secondly, is how to reduce the fluctuations during obstacle avoidance and turning in the EEV algorithm.

ACKNOWLEDGMENT

This work was supported in part by the Researchers Supporting Project number (RSP2023R387), King Saud University, Riyadh, Saudi Arabia, in part by the Natural Science

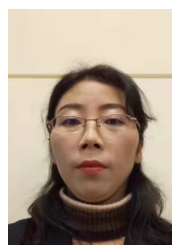
Foundation of Henan Province of China(No. 202300410283), in part by the Science and Technology Program of Henan Province of China(No. 202102210336) and in part by the Key R&D Program in Higher Education of Henan Province of China(No. 20B520018).

REFERENCES

- [1] Muhammad Majid Gulzar, Syed Tahir Hussain Rizvi, Muhammad Yaqoob Javed, Umer Munir, and Haleema Asif. Multi-agent cooperative control consensus: A comparative review. *Electronics*, 7(2):22, 2018.
- [2] Mehmet Serdar Güzel, Vahid Babaei Ajabshir, Panus Nattharith, Emir Cem Gezer, and Serhat Can. A novel framework for multi-agent systems using a decentralized strategy. *Robotica*, 37(4):691–707, 2019.
- [3] Majda Moussa and Giovanni Beltrame. On the robustness of consensus-based behaviors for robot swarms. 14(3):205–231.
- [4] Jun Tang, Gang Liu, and Qingtao Pan. A review on representative swarm intelligence algorithms for solving optimization problems: Applications and trends. *IEEE/CAA Journal of Automatica Sinica*, 8(10):1627–1643, 2021.
- [5] Tugay Alperen Karagüzel, Ali Emre Turgut, and Eliseo Ferrante. Collective gradient perception in a flocking robot swarm. In Marco Dorigo, Thomas Stützle, Maria J. Blesa, Christian Blum, Heiko Hamann, Mary Katherine Heinrich, and Volker Strobel, editors, *Swarm Intelligence*, pages 290–297. Springer International Publishing.
- [6] Andrew Berdahl, Colin J. Torney, Christos C. Ioannou, Jolyon J. Faria, and Iain D. Couzin. Emergent sensing of complex environments by mobile animal groups. 339(6119):574–576. [_eprint: https://www.science.org/doi/pdf/10.1126/science.1225883](https://www.science.org/doi/pdf/10.1126/science.1225883).
- [7] Aalaa I. Hamouda. Cooperative transport in swarm robotics.multi object transportation.
- [8] Palina Bartashevich and Sanaz Mostaghim. Multi-featured collective perception with Evidence Theory: tackling spatial correlations. *Swarm Intelligence*, 15(1):83–110, June 2021.
- [9] EV Melnik, AB Klimenko, and DY Ivanov. A blockchain-based technique for making swarm robots distributed decision. In *Journal of Physics: Conference Series*, volume 1333, page 052013. IOP Publishing, 2019.
- [10] Palina Bartashevich and Sanaz Mostaghim. Ising model as a switch voting mechanism in collective perception. In Paulo Moura Oliveira, Paulo Novais, and Luís Paulo Reis, editors, *Progress in Artificial Intelligence*, pages 617–629, Cham, 2019. Springer International Publishing.
- [11] Vahid Babaei Ajabshir, Mehmet Serdar Guzel, and Erkan Bostanci. A low-cost q-learning-based approach to handle continuous space problems for decentralized multi-agent robot navigation in cluttered environments.

- IEEE Access*, 10:35287–35301, 2022.
- [12] Jun Tang, Xi Chen, Xiaomin Zhu, and Feng Zhu. Dynamic reallocation model of multiple unmanned aerial vehicle tasks in emergent adjustment scenarios. *IEEE Transactions on Aerospace and Electronic Systems*, 59(2):1139–1155, 2023.
- [13] Turk J Elec, Comp Sci, Shahzad Ali, Zenib Khan, and Mahmood Mscs. Investigation on communication aspects of multiple swarm networked robotics. *Turkish Journal of Electrical Engineering and Computer Sciences*, 2019(3), 2019.
- [14] E. Ferrante. Information transfer in a flocking robot swarm. *Ph Thesis*, 2013.
- [15] Alexandros G Dimakis, Soumya Kar, José MF Moura, Michael G Rabbat, and Anna Scaglione. Gossip algorithms for distributed signal processing. *Proceedings of the IEEE*, 98(11):1847–1864, 2010.
- [16] ZJ Liu, M He, ZY Ma, and LF Gu. Uav formation control method based on distributed consistency. *Comput. Eng. Appl.*, 56(23):146–152, 2020.
- [17] Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Neural-swarm: Decentralized close-proximity multirotor control using learned interactions. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3241–3247. IEEE, 2020.
- [18] Yiming Wu, Ming Xu, Ning Zheng, and Xiongxiang He. Event-triggered resilient consensus for multi-agent networks under deception attacks. *IEEE Access*, 8:78121–78129, 2020.
- [19] Dirk Helbing, Illés Farkas, and Tamas Vicsek. Simulating dynamical features of escape panic. *Nature*, 407(6803):487–490, 2000.
- [20] Dirk Helbing, Illes J Farkas, Peter Molnar, and Tamás Vicsek. Simulation of pedestrian crowds in normal and evacuation situations. *Pedestrian and evacuation dynamics*, 21(2):21–58, 2002.
- [21] Guillermo H Goldsztein. A mathematical model of the formation of lanes in crowds of pedestrians moving in opposite directions. *Discrete Dynamics in Nature and Society*, 2015, 2015.
- [22] Mohammad Babar, Ahmad Din, Ohoud Alzamzami, Hanen Karamti, Ahmad Khan, and Muhammad Nawaz. A bacterial foraging based smart offloading for iot sensors in edge computing. *Computers and Electrical Engineering*, 102:108123, 2022.
- [23] Xiwei Liu, Wenlian Lu, and Tianping Chen. Consensus of multi-agent systems with unbounded time-varying delays. *IEEE Transactions on Automatic Control*, 55(10):2396–2401, 2010.
- [24] Guanghui Wen, Zhisheng Duan, Wenwu Yu, and Guangrong Chen. Consensus in multi-agent systems with communication constraints. *International Journal of Robust Nonlinear Control*, 22(2):170–182, 2012.
- [25] Peter Wieland, Jung-Su Kim, Holger Scheu, and Frank Allgöwer. On consensus in multi-agent systems with linear high-order agents. *IFAC Proceedings Volumes*, 41(2):1541–1546, 2008.
- [26] Guangming Xie, Huiyang Liu, Long Wang, and Yingmin Jia. Consensus in networked multi-agent systems via sampled control: fixed topology case. In *2009 American Control Conference*, pages 3902–3907. IEEE, 2009.
- [27] Yanping Gao, Long Wang, Guangming Xie, and Bin Wu. Consensus of multi-agent systems based on sampled-data control. *International Journal of Control*, 82(12):2193–2205, 2009.
- [28] Zhongkui Li, Xiangdong Liu, Mengyin Fu, and Lihua Xie. Global h consensus of multi-agent systems with lipschitz non-linear dynamics. *IET Control Theory & Applications*, 6(13):2041–2048, 2012.
- [29] Mitchell L Neilsen and Masaaki Mizuno. Decentralized consensus protocols. In *Proc. of the 10th Int. Phoenix Conf. on Comp. and Comm*, pages 257–262, 1991.
- [30] Konstantinos I Tsianos and Michael G Rabbat. Distributed consensus and optimization under communication delays. In *2011 49th annual allerton conference on communication, control, and computing (allerton)*, pages 974–982. IEEE, 2011.
- [31] Miloš S Stanković, Dušan M Stipanović, and Srdjan S Stanković. Decentralized consensus based control methodology for vehicle formations in air and deep space. In *Proceedings of the 2010 American Control Conference*, pages 3660–3665. IEEE, 2010.
- [32] Zhirong Qiu, Shuai Liu, and Lihua Xie. Distributed constrained optimal consensus of multi-agent systems. *Automatica*, 68:209–215, 2016.
- [33] Ahmad Din, Muhammed Yousoof Ismail, Babar Shah, Mohammad Babar, Farman Ali, and Siddique Ullah Baig. A deep reinforcement learning-based multi-agent area coverage control for smart agriculture. *Computers and Electrical Engineering*, 101:108089, 2022.
- [34] Reza Olfati-Saber and Richard M Murray. Consensus problems in networks of agents with switching topology and time-delays. *IEEE Transactions on automatic control*, 49(9):1520–1533, 2004.
- [35] Volker Strobel, Eduardo Castelló Ferrer, and Marco Dorigo. Blockchain technology secures robot swarms: a comparison of consensus protocols and their resilience to byzantine robots. *Frontiers in Robotics and AI*, 7:54, 2020.
- [36] Gabriele Valentini, Eliseo Ferrante, and Marco Dorigo. The best-of-n problem in robot swarms: Formalization, state of the art, and novel perspectives. *Frontiers in Robotics and AI*, 4:9, 2017.
- [37] Kelsey Saulnier, David Saldana, Amanda Prorok, George J Pappas, and Vijay Kumar. Resilient flocking for mobile robot teams. *IEEE Robotics and Automation letters*, 2(2):1039–1046, 2017.
- [38] Heath J LeBlanc, Haotian Zhang, Xenofon Koutsoukos, and Shreyas Sundaram. Resilient asymptotic consensus in robust networks. *IEEE Journal on Selected Areas in Communications*, 31(4):766–781, 2013.

- [39] Volker Strobel, Eduardo Castelló Ferrer, and Marco Dorigo. Managing byzantine robots via blockchain technology in a swarm robotics collective decision making scenario. 2018.
- [40] Rui Liu, Fan Jia, Wenhao Luo, Meghan Chandarana, Changjoo Nam, Michael Lewis, and Katia P Sycara. Trust-aware behavior reflection for robot swarm self-healing. In *AAMAS*, pages 122–130, 2019.
- [41] Gabriele Valentini, Davide Brambilla, Heiko Hamann, and Marco Dorigo. Collective perception of environmental features in a robot swarm. In *International Conference on Swarm Intelligence*, pages 65–76. Springer, 2016.
- [42] Andreagiovanni Reina, Gabriele Valentini, Cristian Fernández-Oto, Marco Dorigo, and Vito Trianni. A design pattern for decentralised decision making. *PloS one*, 10(10):e0140950, 2015.
- [43] Mehdi Moussaid, Simon Garnier, Guy Theraulaz, and Dirk Helbing. Collective information processing and pattern formation in swarms, flocks, and crowds. *Topics in Cognitive Science*, 1(3):469–497, 2009.
- [44] Tianyu Wu, Kun Yuan, Qing Ling, Wotao Yin, and Ali H Sayed. Decentralized consensus optimization with asynchrony and delays. *IEEE Transactions on Signal and Information Processing over Networks*, 4(2):293–307, 2017.
- [45] Ragesh K Ramachandran, Nicole Fronda, and Gaurav S Sukhatme. Resilience in multi-robot target tracking through reconfiguration. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4551–4557. IEEE, 2020.
- [46] TV Lakshman and Ashok K Agrawala. Efficient decentralized consensus protocols. *IEEE Transactions on Software Engineering*, (5):600–607, 1986.
- [47] Yu Ye, Hao Chen, Zheng Ma, and Ming Xiao. Decentralized consensus optimization based on parallel random walk. *IEEE Communications Letters*, 24(2):391–395, 2019.
- [48] Gang Chen, Qing Yang, Yongduan Song, and Frank L Lewis. A distributed continuous-time algorithm for non-smooth constrained optimization. *IEEE Transactions on Automatic Control*, 65(11):4914–4921, 2020.
- [49] Benjamin Riviere, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Glas: Global-to-local safe autonomy synthesis for multi-robot motion planning with end-to-end learning. *IEEE Robotics and Automation Letters*, 5(3):4249–4256, 2020.
- [50] Michael Rubenstein, Alejandro Cornejo, and Radhika Nagpal. Programmable self-assembly in a thousand-robot swarm. *Science*, 345(6198):795–799, 2014.



YANG FENGYING is a PhD student in the Department of Computer Science, COMSATS University Islamabad, Abbottabad Campus and School of Computer and Artificial Intelligence of Huanghuai University, Zhumadian, Henan, China. Her research interests include Artificial intelligence and multi-agent systems.



DR. AHMAD DIN is an Associate Professor in Department of AI and Data Science at FASTNUCES Islamabad, Pakistan. He is a Fulbright scholar of University of Georgia, USA. His area of interest includes Artificial Intelligence, Swarm Intelligence, and multi-agent system.



LIU HUICHAO is an associate professor at the School of Computer and Artificial Intelligence of Huanghuai University, Zhumadian, Henan, China. He received his master's degree from Wuhan University in 2010 and his PhD degree from Wuhan University in 2015. His research interests include evolutionary computation and artificial intelligence.



DR. MUHAMMAD BABAR is an assistant professor at Abbottabad University of Science and Technology, Pakistan. He received honor's degree in Computer Science in 2004, following a Master's degree in Data Telecommunication and Networks from University of Salford, UK, in 2010 and received his PhD degree from University of Engineering and Technology Peshawar in 2022. His area of interests includes Edge computing, AIoT, resource management, and computation intelligence.

He is a reviewer and associate editor of reputed international journals in similar domains.



SHAFIQ AHMAD (ashafiq@ksu.edu.sa) received the PhD degree from RMIT University, Melbourne, Australia. He is currently working as an Associate Professor at Industrial Engineering Department, College of Engineering King Saud University Riyadh Saudi Arabia. He has more than two decades working experience both in industry and academia in Australia, Europe and Asia. He has published a research book and several research articles in international journals and refereed conferences.

His research interests are related to smart manufacturing, IIOT and data analytics, multivariate statistical quality control; process monitoring and performance analysis; operations research models and bibliometric network analysis. He is also a certified practitioner in Six Sigma business improvement model.

• • •